

Java

Developer

Interview

Questions —

Complete Master

List

1. Core Java Fundamentals

1. What is Java?
2. What are the main features of Java?
3. Why is Java platform-independent?
4. What is the difference between JDK, JRE, and JVM?
5. What is bytecode?
6. How does Java achieve platform independence?
7. What is the Java compilation process?
8. What is the role of the JVM?
9. What is the difference between Java and C++?
10. Why is Java not considered a purely object-oriented language?
11. What are primitive data types in Java?
12. What is the difference between primitive and reference types?
13. What is type casting?
14. What is implicit vs explicit casting?
15. What are wrapper classes?
16. What is autoboxing and unboxing?
17. What is the difference between `==` and `.equals()`?
18. What are Java operators?

19. What is the difference between `break`, `continue`, and `return`?
 20. What is the difference between `final`, `finally`, and `finalize()`?
 21. What are command-line arguments?
 22. What is the `main()` method signature?
 23. Can we overload the `main()` method?
 24. Why is `main()` static?
 25. Can Java have multiple classes in one file?
 26. What is the default value of instance variables?
 27. What is variable scope?
 28. What is a static variable?
 29. What is a static method?
 30. Can a static method access non-static members?
-

2. Object-Oriented Programming

31. What is OOP?
32. What are the four pillars of OOP?

33. Explain encapsulation.
34. Explain inheritance.
35. Explain polymorphism.
36. Explain abstraction.
37. What is a class?
38. What is an object?
39. What is the difference between class and object?
40. What is a constructor?
41. What are constructor types?
42. Can constructors be overloaded?
43. Can constructors be overridden?
44. Can a constructor be inherited?
45. What is constructor chaining?
46. What is the `this` keyword?
47. What is the `super` keyword?
48. What is method overloading?
49. What is method overriding?
50. Overloading vs overriding?
51. What is compile-time polymorphism?
52. What is runtime polymorphism?
53. What is dynamic method dispatch?
54. What is an abstract class?

55. What is an interface?
56. Abstract class vs interface?
57. Can an abstract class have a constructor?
58. Can an interface have methods with implementation?
59. What are default methods in interfaces?
60. What are static methods in interfaces?
61. Can an interface extend another interface?
62. Can a class implement multiple interfaces?
63. Why doesn't Java support multiple inheritance through classes?
64. How does Java achieve multiple inheritance?
65. What is composition?
66. Composition vs inheritance?
67. What is association?
68. What is aggregation?
69. What is dependency?
70. What is an immutable class?
71. How do you create an immutable class?
72. Why is `String` immutable?
73. What is a marker interface?
74. What is a functional interface?

- 75. What is a nested class?
 - 76. What is an inner class?
 - 77. What is an anonymous inner class?
 - 78. What is a static nested class?
-

3. String Interview Questions

- 79. Why is String immutable in Java?
- 80. What is the String pool?
- 81. What is String interning?
- 82. String VS StringBuilder?
- 83. StringBuilder VS StringBuffer?
- 84. Why is String final?
- 85. What happens when you create a String using `new`?
- 86. What happens when you create a String using a literal?
- 87. How many objects are created by `new String("Java")`?
- 88. What is the difference between `equals()` and `equalsIgnoreCase()`?
- 89. How do you reverse a String?

90. How do you check whether a String is a palindrome?
 91. How do you count characters in a String?
 92. How do you find duplicate characters?
 93. How do you remove duplicate characters?
 94. How do you find the first non-repeated character?
 95. How do you count words in a String?
 96. How do you check whether two Strings are anagrams?
 97. How do you reverse words in a sentence?
 98. How do you find the longest substring without repeating characters?
 99. How do you find the most frequent character?
 100. How do you remove spaces from a String?
-

4. Arrays

101. What is an array?
102. What are the advantages and disadvantages of arrays?
103. Array vs ArrayList?

104. How is an array stored in memory?
 105. Can array size be changed after creation?
 106. What is a multidimensional array?
 107. How do you copy an array?
 108. `Arrays.copyOf()` VS `System.arraycopy()`?
 109. How do you sort an array?
 110. How do you find the largest element?
 111. How do you find the second-largest element?
 112. How do you remove duplicates from an array?
 113. How do you reverse an array?
 114. How do you find missing numbers?
 115. How do you find duplicate numbers?
 116. How do you find common elements between arrays?
 117. How do you merge two arrays?
 118. How do you rotate an array?
 119. How do you find the maximum subarray sum?
 120. How do you find two numbers whose sum equals a target?
-

5. Collections Framework

121. What is the Java Collections Framework?
122. What is the difference between Collection and Collections?
123. List vs Set vs Map?
124. ArrayList vs LinkedList?
125. ArrayList vs Vector?
126. HashSet vs LinkedHashSet?
127. HashSet vs TreeSet?
128. HashMap vs Hashtable?
129. HashMap vs LinkedHashMap?
130. HashMap vs TreeMap?
131. What is a Queue?
132. What is a Deque?
133. Stack vs Queue?
134. What is PriorityQueue?
135. What is ConcurrentHashMap?
136. What is an Iterator?
137. Iterator vs ListIterator?
138. What is Iterable?
139. What is fail-fast behavior?
140. What is fail-safe behavior?
141. What is ConcurrentModificationException?

142. How does ArrayList work internally?
143. How does LinkedList work internally?
144. How does HashSet work internally?
145. How does HashMap work internally?
146. What is a HashMap bucket?
147. What is hashing?
148. What is a hash collision?
149. How does HashMap handle collisions?
150. What is load factor?
151. What is initial capacity?
152. What happens when HashMap reaches its threshold?
153. Why should `equals()` and `hashCode()` be consistent?
154. What happens if two objects have the same `hashCode`?
155. Can HashMap contain null keys?
156. Can HashSet contain null?
157. How do you make a collection immutable?
158. What is `Collections.unmodifiableList()`?
159. What is `List.of()`?

160. What is the difference between mutable and immutable collections?
-

6. Exception Handling

161. What is an exception?
162. Error vs exception?
163. Checked vs unchecked exceptions?
164. What is `Throwable`?
165. What is the exception hierarchy?
166. What is `try-catch`?
167. What is `finally`?
168. Can `finally` execute without `catch`?
169. Can `finally` block be skipped?
170. What is `try-with-resources`?
171. What is `AutoCloseable`?
172. `throw` vs `throws`?
173. Can we throw checked exceptions?
174. Can we have multiple `catch` blocks?
175. What is multi-catch?
176. What happens if an exception occurs in `finally`?

- 177. Can we catch `Exception`?
 - 178. What is a custom exception?
 - 179. How do you create a custom checked exception?
 - 180. How do you create a custom unchecked exception?
 - 181. What is exception propagation?
 - 182. What is stack trace?
 - 183. What is `NullPointerException`?
 - 184. What is `ClassCastException`?
 - 185. What is `NumberFormatException`?
 - 186. What is `ArrayIndexOutOfBoundsException`?
 - 187. What is `ConcurrentModificationException`?
-

7. Java 8+ Features

- 188. What are the major features introduced in Java 8?
- 189. What is a lambda expression?
- 190. What is a functional interface?
- 191. What is the `@FunctionalInterface` annotation?
- 192. What is a Predicate?
- 193. What is a Consumer?
- 194. What is a Supplier?

195. What is a Function?
196. Predicate vs Function?
197. Consumer vs Supplier?
198. What is a method reference?
199. What is the Stream API?
200. Stream vs Collection?
201. What is `map()`?
202. What is `filter()`?
203. What is `flatMap()`?
204. What is `reduce()`?
205. What is `collect()`?
206. What is `forEach()`?
207. What is `sorted()`?
208. What is `distinct()`?
209. What is `limit()`?
210. What is `skip()`?
211. What is lazy evaluation in streams?
212. Intermediate vs terminal operations?
213. What is a parallel stream?
214. Sequential stream vs parallel stream?
215. What is Optional?
216. Why was Optional introduced?

- 217. `orElse()` VS `orElseGet()`?
 - 218. What is `orElseThrow()`?
 - 219. What are default interface methods?
 - 220. What are static interface methods?
 - 221. What is the Date and Time API?
 - 222. `LocalDate` VS `LocalDateTime`?
 - 223. What is `ZonedDateTime`?
 - 224. What is the difference between Java 8, 11, 17, 21, and newer LTS releases?
-

8. Multithreading & Concurrency

- 225. What is a thread?
- 226. Process vs thread?
- 227. How do you create a thread?
- 228. Thread class vs Runnable?
- 229. Runnable vs Callable?
- 230. What is `ExecutorService`?
- 231. What is a thread pool?

- 232. Why use thread pools?
- 233. What is synchronization?
- 234. What is a synchronized method?
- 235. What is a synchronized block?
- 236. What is a lock?
- 237. `synchronized` vs `Lock`?
- 238. What is `ReentrantLock`?
- 239. What is `ReadWriteLock`?
- 240. What is `volatile`?
- 241. `volatile` vs `synchronized`?
- 242. What is atomicity?
- 243. What is visibility?
- 244. What is race condition?
- 245. What is deadlock?
- 246. How can deadlocks be prevented?
- 247. What is livelock?
- 248. What is starvation?
- 249. What is thread safety?
- 250. What is `ThreadLocal`?
- 251. What is `CountDownLatch`?
- 252. What is `CyclicBarrier`?
- 253. What is `Semaphore`?

- 254. What is BlockingQueue?
 - 255. What is CompletableFuture?
 - 256. Future vs CompletableFuture?
 - 257. What is ForkJoinPool?
 - 258. What is parallelism?
 - 259. What is the Java Memory Model?
 - 260. What is happens-before relationship?
-

9. JVM & Memory Management

- 261. What is JVM architecture?
- 262. What are JVM memory areas?
- 263. What is Heap?
- 264. What is Stack?
- 265. What is Metaspace?
- 266. What is the Program Counter Register?
- 267. What is native method stack?
- 268. Heap vs Stack?
- 269. What is garbage collection?

- 270. How does Java Garbage Collection work?
 - 271. What makes an object eligible for GC?
 - 272. What is a memory leak in Java?
 - 273. What is `OutOfMemoryError`?
 - 274. What is `StackOverflowError`?
 - 275. What are strong, weak, soft, and phantom references?
 - 276. What is a GC root?
 - 277. What is Stop-The-World?
 - 278. What are generational garbage collectors?
 - 279. What is G1 GC?
 - 280. What are modern low-latency garbage collectors?
 - 281. How would you investigate high memory usage?
 - 282. How would you investigate frequent GC?
 - 283. What JVM options are commonly used for memory tuning?
-

10. Java I/O

- 284. What is Java I/O?
- 285. `InputStream` vs `Reader`?

- 286. OutputStream vs Writer?
- 287. BufferedReader vs FileReader?
- 288. BufferedWriter vs FileWriter?
- 289. How do you read a file in Java?
- 290. How do you write to a file?
- 291. What is serialization?
- 292. What is deserialization?
- 293. What is Serializable?
- 294. What is serialVersionUID?
- 295. What is transient?
- 296. What is Externalizable?
- 297. Serialization vs Externalization?
- 298. What is Java NIO?
- 299. NIO vs traditional I/O?
- 300. What are Path and Files?
- 301. What is a Channel?
- 302. What is a Buffer?
- 303. What is a Selector?

11. JDBC & Database

304. What is JDBC?
305. What are the steps to connect Java to a database?
306. What is a JDBC Driver?
307. Statement vs PreparedStatement?
308. PreparedStatement vs CallableStatement?
309. Why use PreparedStatement?
310. What is SQL injection?
311. How does PreparedStatement prevent SQL injection?
312. What is ResultSet?
313. What are ResultSet types?
314. What is a database transaction?
315. What are ACID properties?
316. What is commit?
317. What is rollback?
318. What is auto-commit?
319. What is connection pooling?
320. Why is connection pooling important?
321. What is batch processing in JDBC?
322. What is a stored procedure?
323. How do you handle database exceptions?
324. How do you optimize JDBC performance?

12. Hibernate / JPA

- 325. What is Hibernate?
- 326. What is JPA?
- 327. Hibernate vs JPA?
- 328. What is ORM?
- 329. What are the advantages of ORM?
- 330. What is an Entity?
- 331. What is `@Entity`?
- 332. What is `@Id`?
- 333. What is `@GeneratedValue`?
- 334. What is `EntityManager`?
- 335. `EntityManager` vs Hibernate Session?
- 336. What are entity lifecycle states?
- 337. Transient vs persistent vs detached?
- 338. What is persistence context?
- 339. What is lazy loading?
- 340. What is eager loading?
- 341. Lazy vs eager loading?
- 342. What is the N+1 query problem?

- 343. How do you solve N+1?
 - 344. What is JPQL?
 - 345. JPQL vs SQL?
 - 346. What is Criteria API?
 - 347. What is @OneToOne?
 - 348. What is @OneToMany?
 - 349. What is @ManyToOne?
 - 350. What is @ManyToMany?
 - 351. What is cascading?
 - 352. What is orphan removal?
 - 353. What is first-level cache?
 - 354. What is second-level cache?
 - 355. What is optimistic locking?
 - 356. What is pessimistic locking?
 - 357. What is @Version?
 - 358. What is dirty checking?
 - 359. What is Hibernate session management?
-

13. Spring Framework

- 360. What is Spring?

361. Why is Spring used?
362. What is dependency injection?
363. What is inversion of control?
364. DI vs IoC?
365. What is a Spring Bean?
366. What is ApplicationContext?
367. BeanFactory vs ApplicationContext?
368. What is component scanning?
369. What is @Component?
370. What is @Service?
371. What is @Repository?
372. What is @Controller?
373. What is @RestController?
374. @Component VS @Service VS @Repository?
375. What is @Autowired?
376. Constructor injection vs field injection?
377. Why is constructor injection preferred?
378. What is @Qualifier?
379. What is @Primary?
380. What are bean scopes?
381. Singleton vs prototype?
382. What is bean lifecycle?

- 383. What are `@PostConstruct` and `@PreDestroy`?
 - 384. What is Spring AOP?
 - 385. What is an aspect?
 - 386. What is advice?
 - 387. What is a pointcut?
 - 388. What is a join point?
 - 389. What is proxy-based AOP?
 - 390. What is Spring transaction management?
 - 391. What does `@Transactional` do?
 - 392. What are transaction propagation modes?
 - 393. What are transaction isolation levels?
-

14. Spring Boot

- 394. What is Spring Boot?
- 395. Spring vs Spring Boot?
- 396. Advantages of Spring Boot?
- 397. What is auto-configuration?
- 398. What is `@SpringBootApplication`?
- 399. What does `@SpringBootApplication` contain?
- 400. What is Spring Boot Starter?

- 401. What is `application.properties`?
 - 402. What is `application.yml`?
 - 403. How do Spring profiles work?
 - 404. What is `@Configuration`?
 - 405. What is `@Bean`?
 - 406. What is Actuator?
 - 407. What is Spring Boot DevTools?
 - 408. How do you configure a database in Spring Boot?
 - 409. How do you handle environment-specific configuration?
 - 410. How do you create a REST API using Spring Boot?
 - 411. How do you handle exceptions globally?
 - 412. What is `@ControllerAdvice`?
 - 413. What is `@ExceptionHandler`?
 - 414. What is validation in Spring Boot?
 - 415. What are `@Valid` and `@Validated`?
 - 416. How do you implement pagination?
 - 417. How do you implement sorting?
 - 418. How do you implement file upload?
 - 419. How do you configure logging?
 - 420. How do you monitor a Spring Boot application?
-

15. REST API

- 421. What is REST?
- 422. REST vs SOAP?
- 423. What are REST principles?
- 424. What is an HTTP method?
- 425. GET vs POST?
- 426. PUT vs PATCH?
- 427. DELETE vs GET?
- 428. What are HTTP status codes?
- 429. What does 200 mean?
- 430. 201 vs 202?
- 431. 400 vs 401 vs 403?
- 432. 404 vs 409?
- 433. 500 vs 502 vs 503?
- 434. What is idempotency?
- 435. Which HTTP methods are idempotent?
- 436. What is statelessness?
- 437. What is content negotiation?
- 438. What is JSON?
- 439. JSON vs XML?
- 440. What is API versioning?

- 441. What is pagination?
 - 442. What is rate limiting?
 - 443. What is CORS?
 - 444. What is authentication?
 - 445. What is authorization?
 - 446. How do you secure REST APIs?
 - 447. What is JWT?
 - 448. How does JWT authentication work?
 - 449. Access token vs refresh token?
 - 450. How do you design a production-ready REST API?
-

16. Spring Security

- 451. What is Spring Security?
- 452. Authentication vs authorization?
- 453. What is SecurityFilterChain?
- 454. What is a security filter?
- 455. What is JWT authentication?
- 456. How does OAuth 2.0 work?
- 457. What is OpenID Connect?
- 458. OAuth 2.0 vs JWT?

- 459. What is CSRF?
 - 460. What is CORS?
 - 461. How do you implement role-based authorization?
 - 462. What is `@PreAuthorize`?
 - 463. What is password hashing?
 - 464. Why should passwords not be encrypted directly?
 - 465. What is BCrypt?
 - 466. How do you secure endpoints?
 - 467. How do you implement refresh tokens?
 - 468. How do you handle expired tokens?
 - 469. How do you prevent common API security vulnerabilities?
-

17. Microservices

- 470. What are microservices?
- 471. Monolith vs microservices?
- 472. Advantages of microservices?
- 473. Disadvantages of microservices?
- 474. What is service discovery?
- 475. What is an API Gateway?

476. What is load balancing?
477. What is circuit breaker?
478. What is retry?
479. What is timeout?
480. What is bulkhead pattern?
481. What is distributed tracing?
482. What is centralized logging?
483. What is configuration management?
484. What is Spring Cloud?
485. What is OpenFeign?
486. RestTemplate vs WebClient?
487. What is asynchronous communication?
488. Kafka vs REST?
489. What is eventual consistency?
490. What is distributed transaction?
491. What is Saga pattern?
492. What is idempotency in microservices?
493. How do microservices communicate?
494. How do you secure microservices?
495. How do you monitor microservices?
496. How would you design a scalable microservices architecture?

18. Kafka & Messaging

- 497. What is Apache Kafka?
- 498. Why use Kafka?
- 499. Kafka vs RabbitMQ?
- 500. What is a Kafka topic?
- 501. What is a partition?
- 502. What is an offset?
- 503. What is a producer?
- 504. What is a consumer?
- 505. What is a consumer group?
- 506. How does Kafka achieve scalability?
- 507. How does Kafka achieve fault tolerance?
- 508. What is replication factor?
- 509. What is ISR?
- 510. What is message ordering?
- 511. What is consumer lag?
- 512. How do you handle duplicate messages?
- 513. What is at-least-once delivery?
- 514. What is at-most-once delivery?

- 515. What is exactly-once semantics?
 - 516. How do you retry failed messages?
 - 517. What is a dead-letter topic?
 - 518. How would you integrate Kafka with Spring Boot?
-

19. Testing

- 519. What is unit testing?
- 520. What is integration testing?
- 521. Unit vs integration testing?
- 522. What is JUnit?
- 523. JUnit 4 vs JUnit 5?
- 524. What is Mockito?
- 525. Mock vs Spy?
- 526. Mock vs Stub?
- 527. What is @Mock?
- 528. What is @InjectMocks?
- 529. What is @MockBean?
- 530. How do you test REST APIs?
- 531. What is MockMvc?
- 532. What is Testcontainers?

- 533. What is test coverage?
 - 534. What is TDD?
 - 535. What is BDD?
 - 536. How do you test exception scenarios?
 - 537. How do you test database interactions?
 - 538. How do you write maintainable unit tests?
-

20. Design Patterns

- 539. What is a design pattern?
- 540. What is Singleton pattern?
- 541. How do you implement thread-safe Singleton?
- 542. What is Factory pattern?
- 543. Abstract Factory vs Factory?
- 544. What is Builder pattern?
- 545. Why use Builder?
- 546. What is Prototype pattern?
- 547. What is Adapter pattern?
- 548. What is Decorator pattern?
- 549. What is Proxy pattern?
- 550. What is Facade pattern?

- 551. What is Strategy pattern?
 - 552. What is Observer pattern?
 - 553. What is Template Method pattern?
 - 554. What is Chain of Responsibility?
 - 555. What is Command pattern?
 - 556. What is Dependency Injection as a design principle?
 - 557. Which design patterns have you used in Spring Boot?
-

21. SOLID Principles

- 558. What are SOLID principles?
- 559. What is Single Responsibility Principle?
- 560. What is Open/Closed Principle?
- 561. What is Liskov Substitution Principle?
- 562. What is Interface Segregation Principle?
- 563. What is Dependency Inversion Principle?
- 564. Give a real-world example of SRP.
- 565. Give an example of OCP.
- 566. How does Spring implement Dependency Inversion?

567. How do SOLID principles improve maintainability?

22. Java Coding Questions

- 568. Reverse a String.
- 569. Reverse an integer.
- 570. Check whether a number is palindrome.
- 571. Check whether a String is palindrome.
- 572. Find factorial of a number.
- 573. Generate Fibonacci series.
- 574. Check whether a number is prime.
- 575. Print prime numbers in a range.
- 576. Find Armstrong numbers.
- 577. Find GCD and LCM.
- 578. Swap two numbers without a third variable.
- 579. Find the largest number in an array.
- 580. Find the second-largest number.
- 581. Find duplicate elements.
- 582. Find missing number.
- 583. Remove duplicates.
- 584. Sort an array without built-in sorting.

- 585. Merge two sorted arrays.
 - 586. Rotate an array.
 - 587. Find maximum subarray sum.
 - 588. Find two-sum pairs.
 - 589. Find three-sum combinations.
 - 590. Count character frequency.
 - 591. Find duplicate characters.
 - 592. Find first non-repeated character.
 - 593. Check whether two Strings are anagrams.
 - 594. Reverse words in a sentence.
 - 595. Find longest word.
 - 596. Find longest substring without repeated characters.
 - 597. Count vowels and consonants.
 - 598. Remove whitespace.
 - 599. Find common elements between arrays.
 - 600. Find intersection and union of arrays.
-

23. Stream API Coding

Questions

601. Find even numbers using streams.
602. Find odd numbers using streams.
603. Find maximum value using streams.
604. Find minimum value using streams.
605. Find sum using streams.
606. Find average using streams.
607. Remove duplicates using streams.
608. Sort a list using streams.
609. Sort in descending order.
610. Find duplicate elements using streams.
611. Find first non-repeated character using streams.
612. Count character frequency using streams.
613. Group employees by department.
614. Find highest-paid employee.
615. Find second-highest salary.
616. Find employees earning above a threshold.
617. Convert List to Map.
618. Partition numbers into even and odd.
619. Find the longest String.
620. Find top N values.
621. Flatten nested lists using `flatMap()`.
622. Group data using `Collectors.groupingBy()`.

- 623. Use `partitioningBy()`.
 - 624. Explain when streams should not be used.
-

24. Java Scenario-Based Questions

- 625. Your API is slow. How would you investigate it?
- 626. Your application has high CPU usage. What would you check?
- 627. Your application has high memory usage. How would you debug it?
- 628. How would you investigate a memory leak?
- 629. How would you debug a deadlock?
- 630. Your database queries are slow. What would you do?
- 631. Your REST API receives thousands of requests per second. How would you scale it?
- 632. How would you design a thread-safe cache?
- 633. How would you handle duplicate API requests?
- 634. How would you implement rate limiting?

- 635. How would you design a retry mechanism?
 - 636. How would you prevent cascading failures?
 - 637. How would you handle Kafka consumer failures?
 - 638. How would you design an idempotent API?
 - 639. How would you handle distributed transactions?
 - 640. How would you migrate a monolith to
microservices?
 - 641. How would you secure sensitive API endpoints?
 - 642. How would you handle backward compatibility?
 - 643. How would you deploy a Spring Boot application?
 - 644. How would you monitor a production Java
application?
 - 645. How would you troubleshoot intermittent
production failures?
-

25. System Design Questions for Java Developers

- 646. Design an e-commerce application.
- 647. Design an online payment system.

- 648. Design a URL shortener.
 - 649. Design a food-delivery application.
 - 650. Design a ride-sharing system.
 - 651. Design a notification service.
 - 652. Design an authentication service.
 - 653. Design a file-storage service.
 - 654. Design a logging system.
 - 655. Design a distributed cache.
 - 656. Design a rate limiter.
 - 657. Design an API Gateway.
 - 658. Design a ticket-booking system.
 - 659. Design an online banking system.
 - 660. Design a chat application.
 - 661. Design a video-streaming platform.
 - 662. Design a product-search service.
 - 663. Design an order-management system.
 - 664. Design an inventory system.
 - 665. Design a scalable Java microservices platform.
-

26. HR & Behavioral

Questions

- 666. Tell me about yourself.
- 667. Explain your Java experience.
- 668. Explain your current project.
- 669. What is your role in your current project?
- 670. Why did you choose Java?
- 671. Why do you want to change jobs?
- 672. What was your most challenging Java project?
- 673. Describe a difficult production issue you solved.
- 674. Tell me about a performance optimization you implemented.
- 675. Tell me about a bug you solved.
- 676. How do you handle deadlines?
- 677. How do you handle disagreements with teammates?
- 678. How do you review code?
- 679. How do you ensure code quality?
- 680. How do you learn new Java technologies?
- 681. What Java feature do you like most?
- 682. What are your strengths as a Java developer?

- 683. What is your weakness?
 - 684. Why should we hire you?
 - 685. Where do you see yourself in five years?
-

27. Senior Java Developer Questions

- 686. How do you design highly scalable Java applications?
- 687. How do you optimize JVM performance?
- 688. How do you choose between synchronous and asynchronous processing?
- 689. How do you design fault-tolerant services?
- 690. How do you manage distributed transactions?
- 691. How do you handle eventual consistency?
- 692. How do you design highly available systems?
- 693. How do you investigate production memory leaks?
- 694. How do you tune garbage collection?
- 695. How do you optimize database access?
- 696. How do you identify bottlenecks?

- 697. How do you design thread-safe components?
 - 698. How do you prevent race conditions?
 - 699. How do you design resilient APIs?
 - 700. How do you handle observability?
 - 701. How do you approach code reviews?
 - 702. How do you introduce architectural improvements without breaking existing systems?
 - 703. How do you migrate legacy Java applications?
 - 704. How do you design microservices boundaries?
 - 705. How do you make Java applications cloud-ready?
-

Most Important Topics to Master

For a **Java Developer** interview, prioritize these areas:

1. Core Java
2. OOP
3. String handling
4. Collections
5. Exception handling
6. Java 8+ features
7. Streams and Lambda

8. Multithreading
9. JVM and memory management
10. SQL and JDBC
11. Hibernate/JPA
12. Spring
13. Spring Boot
14. REST APIs
15. Spring Security
16. Microservices
17. Kafka
18. JUnit/Mockito
19. Design Patterns
20. SOLID
21. Data Structures & Algorithms
22. System Design
23. Git
24. Docker
25. Cloud basics
26. CI/CD
27. Production debugging
28. Project-based questions
29. Scenario-based questions

30. HR/behavioral questions

Interview priority: For fresher roles, focus heavily on **Core Java + OOP + Collections + Java 8 + SQL + coding**. For 2–5 years, add **Spring Boot + REST + Hibernate + testing + microservices**. For senior roles, expect deep questions on **JVM, concurrency, distributed systems, Kafka, system design, performance, security, observability, and production architecture**.